



Universidade Federal  
do Rio de Janeiro  

---

Escola Politécnica

# EEL891 — Introdução ao Aprendizado de Máquina

## Trabalho 2

### Regressão de Preços de Imóveis

Estimativa de preço de venda por regressão multivariável

**Aluno:** Luiz Felipe Píccoli Cavalini

**Ferramentas:** Python 3.12 (scikit-learn, LightGBM,  
XGBoost, CatBoost, Optuna, SciPy)

Rio de Janeiro, Julho de 2026

# Sumário

|           |                                                                                    |           |
|-----------|------------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introdução</b>                                                                  | <b>1</b>  |
| <b>2</b>  | <b>Análise Exploratória dos Dados</b>                                              | <b>1</b>  |
| 2.1       | Distribuição do preço . . . . .                                                    | 1         |
| 2.2       | Correlações e cardinalidade . . . . .                                              | 2         |
| <b>3</b>  | <b>Tratamento de <i>Outliers</i></b>                                               | <b>3</b>  |
| <b>4</b>  | <b>Transformação do <i>Target</i></b>                                              | <b>3</b>  |
| <b>5</b>  | <b>Engenharia de <i>Features</i></b>                                               | <b>3</b>  |
| <b>6</b>  | <b>Codificação de Variáveis Categóricas</b>                                        | <b>4</b>  |
| 6.1       | <i>One-Hot Encoding</i> . . . . .                                                  | 4         |
| 6.2       | Bairro: <i>Target Encoding</i> com <i>smoothing + frequency encoding</i> . . . . . | 4         |
| 6.2.1     | Abordagens testadas e descartadas . . . . .                                        | 5         |
| <b>7</b>  | <b>Modelagem</b>                                                                   | <b>5</b>  |
| <b>8</b>  | <b>Otimização de Hiperparâmetros</b>                                               | <b>6</b>  |
| 8.1       | Impacto do número de <i>trials</i> . . . . .                                       | 6         |
| <b>9</b>  | <b><i>Ensemble</i>: Diagnóstico de Correlação</b>                                  | <b>7</b>  |
| <b>10</b> | <b>Abordagens de <i>Ensemble</i> Testadas e Descartadas</b>                        | <b>8</b>  |
| <b>11</b> | <b>Resultados</b>                                                                  | <b>9</b>  |
| <b>12</b> | <b>Importância das <i>Features</i></b>                                             | <b>9</b>  |
| <b>13</b> | <b>Conclusões</b>                                                                  | <b>11</b> |
|           | <b>Referências</b>                                                                 | <b>12</b> |

## 1 Introdução

O presente trabalho consiste na construção de um modelo de regressão multivariável para estimar o preço de venda de imóveis residenciais. A partir de dados históricos de 4.683 imóveis de treino, acompanhados do preço efetivamente praticado, e de 20 características descritivas (tipo, localização, área, quartos, vagas, amenidades, entre outras), o objetivo é prever o preço de 2.000 novos imóveis cujos preços são desconhecidos.

A métrica de avaliação é o **RMSPE** (*Root Mean Square Percentage Error*), definida como

$$\text{RMSPE} = \sqrt{\frac{1}{n} \sum_{i=1}^n \left( \frac{y_i - \hat{y}_i}{y_i} \right)^2} \quad (1)$$

que penaliza o erro **percentual**, não o erro absoluto: errar R\$ 100 mil num imóvel de R\$ 200 mil é muito pior do que errar R\$ 100 mil num imóvel de R\$ 2 milhões. As previsões são submetidas na plataforma Kaggle, que compara automaticamente com o gabarito oculto.

O pipeline adotado segue as etapas clássicas: análise exploratória, pré-processamento (incluindo tratamento cuidadoso de *outliers*), engenharia de *features*, modelagem com múltiplos algoritmos, otimização bayesiana de hiperparâmetros e construção de *ensembles* ponderados. As ferramentas utilizadas foram Python 3.12 com **pandas** [10], **scikit-learn** [1], **LightGBM** [2], **XGBoost** [3], **CatBoost** [4], **Optuna** [5] e **SciPy** [11].<sup>1</sup>

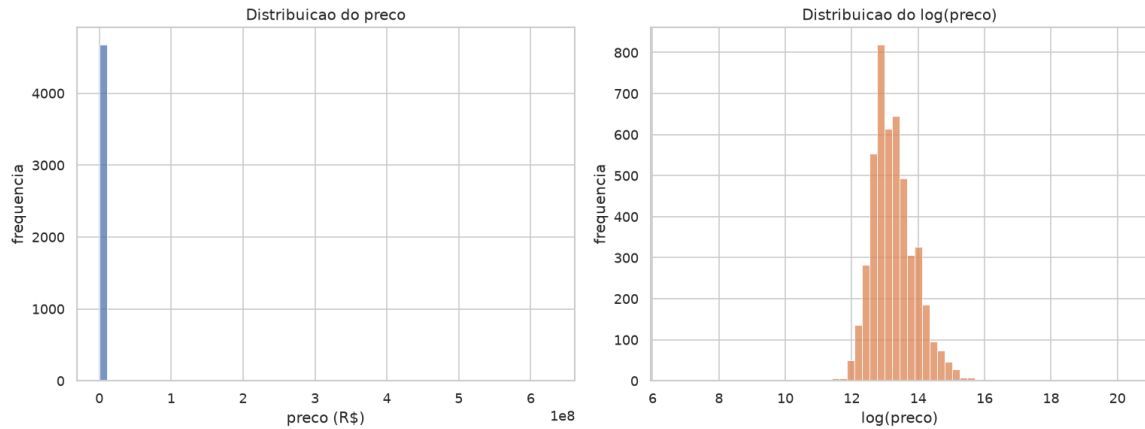
## 2 Análise Exploratória dos Dados

### 2.1 Distribuição do preço

O preço dos imóveis apresenta distribuição fortemente assimétrica à direita, com cauda longa causada por um pequeno número de imóveis de altíssimo valor. Ao aplicar a transformação  $\log(1 + \text{preço})$ , a distribuição se aproxima de uma normal (Figura 1), o que motiva o uso do *target* em escala logarítmica durante o treinamento (Seção 4).

---

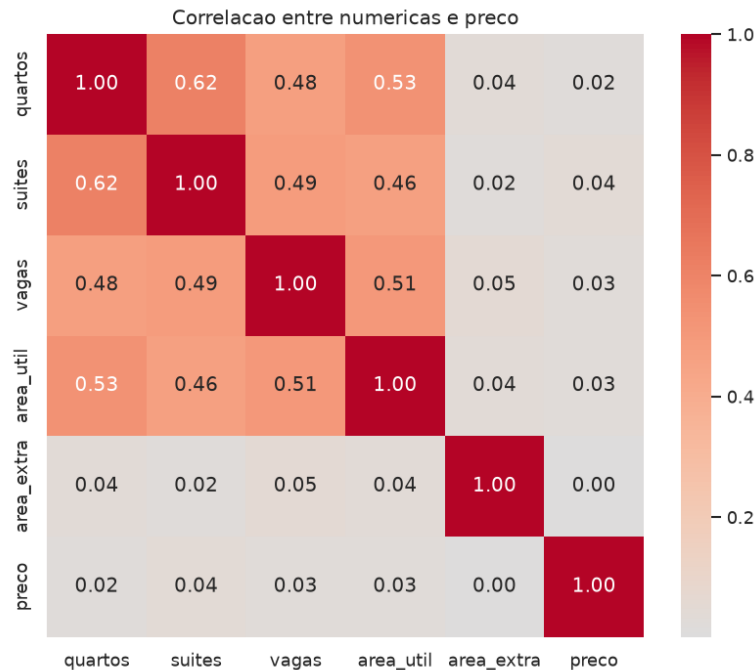
<sup>1</sup>O código-fonte completo, incluindo todos os experimentos descritos neste relatório e o histórico de desenvolvimento, está disponível em <https://github.com/LuizCavalini/ML-RealEstatePricing>.



**Figura 1:** Distribuição do preço bruto (esquerda) e de  $\log(1 + \text{preço})$  (direita).

## 2.2 Correlações e cardinalidade

As correlações lineares das variáveis numéricas (`quartos`, `suites`, `vagas`, `area_util`, `area_extra`) com o preço bruto são todas muito fracas ( $|r| < 0,05$ , Figura 2), o que reforça a necessidade da transformação logarítmica do *target* e de *features* de interação não-lineares (Seção 5).



**Figura 2:** Matriz de correlação entre variáveis numéricas e o preço.

O conjunto possui 4 tipos de imóvel (predominantemente “Apartamento”), 66 bairros distintos no treino (alta cardinalidade, exigindo *encoding* cuidadoso — Seção 6) e 10 amenidades binárias (piscina, churrasqueira, sauna, vista para o mar, entre outras).

### 3 Tratamento de *Outliers*

Esta etapa teve o **maior impacto individual** de todo o trabalho. Os *baselines* iniciais, treinados sem qualquer tratamento de *outliers*, apresentaram RMSPE entre **196% e 4867%** — um sinal inequívoco de que alguns poucos imóveis com preços absurdos estavam dominando a métrica percentual.

Aplicando o critério  $\text{IQR} \times 3$  sobre  $\log(1 + \text{preço})$  no conjunto de treino, foram identificados 4 imóveis: dois de altíssimo valor (R\$ 630 milhões e R\$ 340 milhões, quase certamente erros de digitação), um de R\$ 65 milhões, e um de apenas R\$ 750 (provavelmente com dígitos faltando). A remoção desses 4 imóveis (de 4.683) derrubou o RMSPE do CatBoost de 2,083 para 0,238 — uma redução de **88%**, com reduções de até 92% para o XGBoost (Tabela 1).

**Tabela 1:** Impacto da remoção de *outliers* ( $\text{IQR} \times 3$ ) no RMSPE

| Modelo       | RMSPE com <i>outliers</i> | RMSPE sem <i>outliers</i> |
|--------------|---------------------------|---------------------------|
| CatBoost [4] | 2,083                     | 0,238                     |
| LightGBM [2] | 2,594                     | 0,249                     |
| XGBoost [3]  | 3,369                     | 0,268                     |

Um corte adicional, mais agressivo, no percentil 99 do preço (removendo mais 47 imóveis do treino, que passou de 4.679 para **4.632** amostras) reduziu ainda mais o RMSPE, de 0,2306 para **0,2240**. Em nenhum momento o conjunto de teste foi alterado — a remoção de *outliers* ocorre exclusivamente no treino, para que o modelo não aprenda com dados espúrios.

### 4 Transformação do *Target*

Todo o treinamento foi conduzido sobre  $y = \log(1 + \text{preço})$ , com as previsões convertidas de volta à escala original por  $\text{prêço} = \exp(\hat{y}) - 1$ . A justificativa é direta: como o RMSPE penaliza o erro **percentual**, e o erro percentual no espaço original é aproximadamente equivalente ao erro absoluto no espaço logarítmico, treinar em  $\log(1 + \text{preço})$  faz com que a função de perda usual dos modelos de *gradient boosting* (baseada em erro absoluto/quadrático) já esteja, de forma aproximada, otimizando a métrica de avaliação real.

### 5 Engenharia de *Features*

Foram criadas diversas variáveis a partir dos atributos originais, guiadas pela análise exploratória e pelo conhecimento de domínio imobiliário (Tabela 2).

**Tabela 2:** *Features* criadas e motivação

| <i>Feature</i>                                                                     | Motivação                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>area_total</code>                                                            | Soma de <code>area_util</code> e <code>area_extra</code> — área real disponível no imóvel.                                                                                                                                                          |
| <code>log_area_util</code>                                                         | Reduz assimetria da área, análogo à transformação do <i>target</i> .                                                                                                                                                                                |
| <code>area_por_quarto</code>                                                       | Área relativa ao número de cômodos — captura “amplidão” independente do tamanho total.                                                                                                                                                              |
| <code>total_amenidades</code>                                                      | Soma das 10 amenidades binárias — proxy de padrão do imóvel.                                                                                                                                                                                        |
| <code>categoria_luxo</code>                                                        | Flag: piscina + sauna + vista para o mar $\geq 2$ — combinação típica de imóveis de alto padrão.                                                                                                                                                    |
| <code>preco_m2_estimado</code>                                                     | Mediana de (preço / <code>area_util</code> ) por bairro, com <i>smoothing</i> , multiplicada pela área do imóvel — dá ao modelo uma estimativa de preço já informada por localização e tamanho, em vez de deixá-lo aprender essa interação do zero. |
| <code>quartos_x_area</code> , <code>vagas_x_area</code> , <code>luxo_x_area</code> | Interações entre quantidade (quartos, vagas, amenidades) e área — capturam que o efeito de um cômodo extra difere conforme o tamanho do imóvel.                                                                                                     |
| <code>tipo_x_area</code>                                                           | Interação entre cada categoria de <code>tipo</code> ( <i>one-hot</i> ) e a área — apartamentos e casas valorizam área de forma diferente.                                                                                                           |

## 6 Codificação de Variáveis Categóricas

### 6.1 *One-Hot Encoding*

Aplicado a `tipo` (4 categorias: Apartamento, Casa, Loft, Quitinete).

### 6.2 Bairro: *Target Encoding* com *smoothing* + *frequency encoding*

Com 66 categorias no treino, `bairro` exige uma codificação que capture o efeito de localização sem introduzir *overfitting* excessivo. A abordagem final combina duas técnicas:

- **Target encoding com smoothing** ( $m = 10$ ):  $TE_b = \frac{n_b \cdot \bar{y}_b + m \cdot \bar{y}_{\text{global}}}{n_b + m}$ , onde bairros com poucas amostras são puxados em direção à média global, seguindo a técnica descrita por Micci-Barreca [8].
- **Frequency encoding**: frequência relativa do bairro no treino, como *feature* adi-

cional.

### 6.2.1 Abordagens testadas e descartadas

- **Target encoding sem smoothing** (mediana simples por bairro): RMSPE 0,2355, pior que a combinação *smoothed + frequency* (0,2320).
- **Frequency encoding sozinho** (sem *target encoding*): RMSPE 0,2405, o pior entre as variantes testadas.
- **Leave-One-Out (LOO) encoding**: para cada amostra, a média do bairro é calculada excluindo a própria amostra. RMSPE honesto de **0,3879** — muito pior que o *smoothing*, pois bairros com poucas amostras geram estimativas LOO extremamente ruidosas, sem a regularização que o *smoothing* oferece.
- **Bug de data leakage detectado e corrigido**: a primeira implementação do LOO calculava as estatísticas por bairro **globalmente**, antes da validação cruzada, e só então aplicava a exclusão da própria amostra. Isso vazava informação entre *folds* — duas amostras do mesmo bairro em *folds* de validação diferentes continuavam “se enxergando” pela estatística agregada compartilhada. O resultado era um RMSPE artificialmente baixo de **0,1627**, quase 30% melhor que qualquer outra técnica testada em todo o trabalho — grande demais para ser plausível. A correção recalculou o LOO separadamente dentro de cada *fold*, usando somente o treino daquele *fold*, revelando o verdadeiro desempenho (0,3879, muito pior). Esse episódio ilustra como o vazamento de dados em *target encoding* pode produzir ganhos de validação cruzada enganosamente bons, como também discutido por Hastie et al. [9].

**Conclusão:** a combinação *target encoding* com *smoothing + frequency encoding* foi mantida como codificação final do bairro.

## 7 Modelagem

Todos os modelos foram avaliados com **5-Fold Cross-Validation** sobre o RMSPE, já no conjunto de treino sem os 4 *outliers* extremos (Seção 3). A Tabela 3 apresenta os resultados com hiperparâmetros padrão.

**Tabela 3:** RMSPE CV dos modelos base (após remoção de *outliers*)

| Modelo            | RMSPE CV      | Desvio  |
|-------------------|---------------|---------|
| Regressão Linear  | 3,8762        | 7,1787  |
| Ridge             | 2,4124        | 4,2509  |
| Lasso             | 31,6641       | 40,7961 |
| Random Forest [7] | 0,2556        | 0,0135  |
| Gradient Boosting | 0,2537        | 0,0178  |
| LightGBM [2]      | 0,2486        | 0,0093  |
| XGBoost [3]       | 0,2683        | 0,0185  |
| CatBoost [4]      | <b>0,2380</b> | 0,0131  |

O CatBoost foi o melhor modelo individual (0,238), seguido de perto por LightGBM e XGBoost. Regressão Linear, Ridge e Lasso ficaram muito atrás — esperado, dado que as correlações lineares com o preço são fracas (Figura 2).

## 8 Otimização de Hiperparâmetros

A otimização foi conduzida com **Optuna** [5] (busca bayesiana via TPE), com cada *trial* avaliado por um *split* 80/20 estratificado por quintis de  $\log(1 + \text{preço})$  — garantindo que faixas de preço distintas (baratos, médios, caros) estivessem representadas tanto no treino quanto na validação de cada *trial*.

### 8.1 Impacto do número de *trials*

Foi conduzida uma primeira rodada com 50 *trials* por modelo (RMSPE de validação: LightGBM 0,2144, XGBoost 0,2126, CatBoost 0,2102) e uma rodada estendida com 150/150/80 *trials* para LightGBM/XGBoost/CatBoost respectivamente (RMSPE de validação: 0,2111, 0,2117, 0,2077). No Kaggle, a rodada estendida melhorou o score de **0,2469 para 0,2437**.

Esse resultado é **oposto** ao observado no Trabalho 1 (classificação de inadimplência), onde buscas mais longas de hiperparâmetros geraram *overfitting* ao processo de validação e pioraram o desempenho no Kaggle. A explicação mais provável é o tamanho do conjunto de dados: o Trabalho 1 possuía 20.000 amostras, contra apenas 4.632 aqui — em um espaço amostral menor, um orçamento maior de *trials* tem mais espaço para explorar a região de hiperparâmetros sem necessariamente decorar particularidades de um único *split*.

**Tabela 4:** Hiperparâmetros finais (Optuna, 150/150/80 *trials*, *split* 80/20)

| Modelo   | Hiperparâmetros                                                                                                                                          |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| LightGBM | n_estimators=834, max_depth=8, num_leaves=51,<br>learning_rate=0,0059, min_child_samples=13,<br>subsample=0,44, colsample_bytree=0,46, reg_lambda=0,52   |
| XGBoost  | n_estimators=1036, max_depth=10, learning_rate=0,0054,<br>min_child_weight=19, subsample=0,66,<br>colsample_bytree=0,42, reg_alpha=0,11, reg_lambda=0,64 |
| CatBoost | iterations=653, depth=10, learning_rate=0,0165,<br>l2_leaf_reg=1,17, bagging_temperature=0,49,<br>random_strength=0,0064                                 |

## 9 *Ensemble*: Diagnóstico de Correlação

Esta foi a etapa de maior ganho após o tratamento de *outliers*. O *blending* uniforme dos 3 *boosters* otimizados não superava RMSPE de 0,2220 na validação cruzada, por melhor que fosse cada modelo individual. A causa foi investigada calculando a **matriz de correlação das previsões *out-of-fold* (OOF)** de 8 modelos: os 3 *boosters*, mais Random Forest [7], ExtraTrees, Ridge, ElasticNet e KNN.

**Tabela 5:** Correlação das previsões OOF com o CatBoost

| Modelo        | Correlação com CatBoost (OOF) |
|---------------|-------------------------------|
| XGBoost       | 0,995                         |
| LightGBM      | 0,995                         |
| Random Forest | 0,992                         |
| ExtraTrees    | 0,992                         |
| Ridge         | 0,967                         |
| ElasticNet    | 0,966                         |
| KNN           | 0,962                         |

O diagnóstico foi claro: LightGBM, XGBoost, CatBoost, Random Forest e ExtraTrees — todos algoritmos baseados em árvores sobre as mesmas *features* — apresentam correlação  $\geq 0,99$  entre si, errando quase sempre nos mesmos imóveis. Ridge, ElasticNet e KNN são estruturalmente diferentes (lineares e baseado em vizinhança) e, de fato, bem menos correlacionados ( $\approx 0,96$ ) — porém individualmente muito piores (RMSPE entre 0,267 e 0,276, contra  $\approx 0,22$  dos *boosters*).

Um *blend* uniforme desses modelos fracos com os fortes pioraria o resultado. A solução foi **otimizar os pesos do *blend*** via `scipy.optimize.minimize` [11] (método

SLSQP), com restrições  $w_i \geq 0$  e  $\sum_i w_i = 1$ , minimizando diretamente o RMSPE das previsões OOF ponderadas:

**Tabela 6:** Pesos otimizados do *ensemble* final

| Modelo                        | Peso |
|-------------------------------|------|
| CatBoost                      | 0,39 |
| LightGBM                      | 0,32 |
| XGBoost                       | 0,14 |
| KNN                           | 0,13 |
| Random Forest                 | 0,02 |
| ElasticNet, Ridge, ExtraTrees | 0,00 |

O otimizador extraiu valor real do KNN — individualmente o pior modelo do grupo — justamente por seus erros serem menos correlacionados com os dos *boosters*; um peso pequeno mas não-nulo (0,13) foi suficiente para reduzir o erro do *ensemble* sem deixar o KNN dominar. ElasticNet, Ridge e ExtraTrees foram zerados pelo otimizador, indicando que não agregavam informação incremental relevante. O resultado foi RMSPE de **0,2205** na validação cruzada e **0,2393** no Kaggle — o melhor score de todo o trabalho.

## 10 Abordagens de *Ensemble* Testadas e Descartadas

- **Stacking** com meta-*learner* Ridge sobre as previsões OOF dos 3 *boosters*: RMSPE 0,2221, essencialmente empatado com o *voting* uniforme (0,2220).
- **Blending de submissões** já geradas (média em espaço logarítmico): não trouxe ganho — as submissões tinham correlação  $> 0,998$  entre si, um sintoma da mesma limitação diagnosticada na Seção 9.
- **Feature selection** (remoção de *features* com importância  $< 1\%$  no CatBoost): RMSPE 0,2242, pior que manter todas as *features* (0,2234).
- **10 seeds vs. 5 seeds** no *blend* multi-*seed*: RMSPE idêntico (0,2223 em ambos) — a redução de variância satura rapidamente, não compensando o custo computacional extra.
- **Regularização L2 dos pesos do ensemble** ( $\text{RMSPE} + \lambda \sum w_i^2$ ,  $\lambda \in \{0; 0,001; 0,01; 0,1\}$ ):  $\lambda = 0$  venceu; a penalização só piorou o resultado, sugerindo que a concentração de peso nos melhores modelos não era um artefato de *overfitting* ao OOF.
- **Features de palavras-chave** extraídas do campo de texto livre diferenciais (“gourmet”, “mármore”, etc.): sem ganho algum, pois o campo na verdade usa

um vocabulário fixo de combinações das 10 amenidades binárias, não texto livre genuíno.

- **Optuna direto no 5-fold CV** (em vez do *split* 80/20) para o CatBoost: RMSPE 0,2237, contra 0,2235 do *split* simples — sem ganho, apesar do custo computacional 5 vezes maior por *trial*.
- **Multi-*seed* nos pesos otimizados** (*pool* de 14 modelos: 3 *boosters*  $\times$  3 *seeds* + 5 modelos diversos): o RMSPE de validação cruzada melhorou para 0,2199, mas o Kaggle **piorou** para 0,2398 — o otimizador concentrou quase todo o peso em *seeds* específicas de cada *booster*, um sinal de *overfitting* ao conjunto OOF que não se confirmou no teste real, reforçando a importância de desconfiar de ganhos marginais de validação cruzada.

## 11 Resultados

A Tabela 7 resume a evolução das principais submissões ao Kaggle ao longo do trabalho.

**Tabela 7:** Evolução das submissões ao Kaggle

| Descrição                                                                          | RMSPE Kaggle  |
|------------------------------------------------------------------------------------|---------------|
| Baseline ( <i>blend/voting</i> inicial)                                            | 0,2535        |
| + <i>encoding</i> smoothed, <i>features</i> de interação, corte de <i>outliers</i> | 0,2479        |
| + re-otimização Optuna no dataset limpo                                            | 0,2469        |
| + Optuna estendido (150/150/80 <i>trials</i> )                                     | 0,2437        |
| + <b><i>ensemble</i> ponderado diverso (pesos otimizados)</b>                      | <b>0,2393</b> |
| (teste) multi- <i>seed</i> nos pesos, 14 modelos                                   | 0,2398        |

O melhor score obtido foi **0,2393**, uma melhoria de **5,6%** em relação ao *base-line* inicial (0,2535). A etapa de maior impacto isolado foi o tratamento de *outliers* (Seção 3); a etapa de maior impacto *fino* foi o diagnóstico de correlação do *ensemble* (Seção 9).

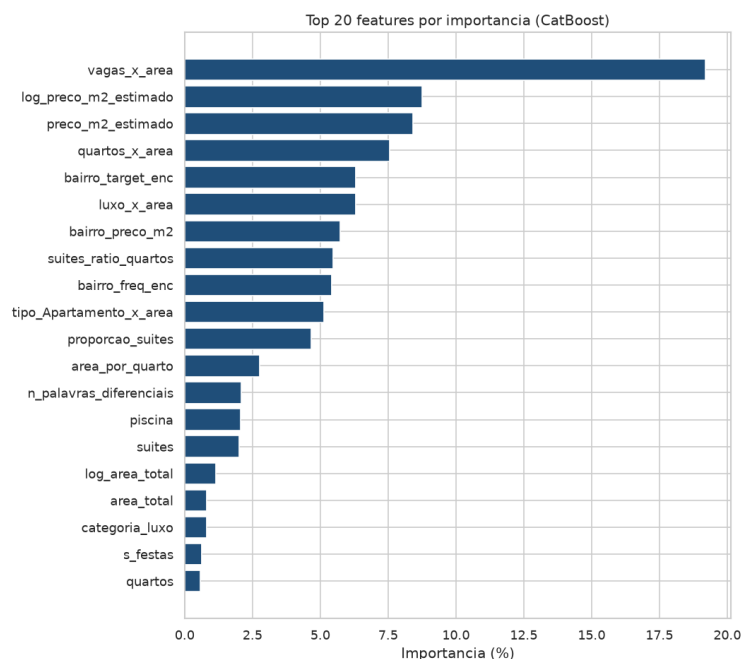
## 12 Importância das *Features*

A Figura 3 apresenta a importância das 20 *features* mais relevantes segundo o CatBoost otimizado. A Tabela 8 lista as 10 primeiras.

**Tabela 8:** Top 10 *features* por importância (CatBoost)

| #  | Feature                               | Importância |
|----|---------------------------------------|-------------|
| 1  | vagas_x_area (engenharia)             | 19,2%       |
| 2  | log_preco_m2_estimado (engenharia)    | 8,8%        |
| 3  | preco_m2_estimado (engenharia)        | 8,4%        |
| 4  | quartos_x_area (engenharia)           | 7,6%        |
| 5  | bairro_target_enc ( <i>encoding</i> ) | 6,3%        |
| 6  | luxo_x_area (engenharia)              | 6,3%        |
| 7  | bairro_preco_m2 (engenharia)          | 5,7%        |
| 8  | suites_ratio_quartos (engenharia)     | 5,5%        |
| 9  | bairro_freq_enc ( <i>encoding</i> )   | 5,4%        |
| 10 | tipo_Apartamento_x_area (engenharia)  | 5,1%        |

Destaca-se que **todas** as 10 *features* mais importantes foram criadas na etapa de engenharia de *features* ou de *encoding* (Seções 5 e 6) — nenhuma variável original bruta aparece no topo, validando fortemente o investimento nessas etapas. A variável **vagas\_x\_area** isoladamente concentra quase 20% (19,2%) da importância total, sugerindo que a interação entre vagas de garagem e área do imóvel é o sinal mais forte disponível para prever o preço. Chama atenção também que **três** das dez posições (**log\_preco\_m2\_estimado**, **preco\_m2\_estimado** e **bairro\_preco\_m2**) pertencem à família de *features* de preço por m<sup>2</sup> descrita na Seção 5 — reforçando que essa estimativa informada por localização e tamanho é, de fato, um dos sinais mais úteis do modelo.

**Figura 3:** Top 20 *features* por importância no CatBoost otimizado.

## 13 Conclusões

1. **A transformação logarítmica do *target*** é essencial quando a métrica de avaliação é percentual (RMSPE): treinar em  $\log(1 + \text{preço})$  aproxima a função de perda usual dos modelos da métrica real de avaliação.
2. **O tratamento de *outliers* teve o maior impacto individual** de todo o trabalho: remover apenas 4 de 4.683 amostras (0,09% do treino) derrubou o RMSPE de aproximadamente 200% para cerca de 24%.
3. ***Target encoding* requer regularização (*smoothing*)**: a versão pura (*Leave-One-Out*) mostrou-se ruidosa demais em categorias esparsas, e uma primeira implementação continha um *bug* de vazamento de dados entre *folds* que produzia ganhos de validação enganosamente otimistas — um lembrete de que resultados bons demais merecem investigação antes de serem aceitos.
4. **A correlação entre modelos limita o ganho de *ensemble***: 5 modelos baseados em árvores, por mais distintos que sejam em sua implementação (LightGBM, XGBoost, CatBoost, Random Forest, ExtraTrees), apresentaram correlação  $\geq 0,99$  entre si e, portanto, pouco se complementaram.
5. **Diversidade estrutural com pesos otimizados quebrou esse teto**: um modelo individualmente fraco (KNN, RMSPE 0,276) melhorou o *ensemble* final porque seus erros ocorrem em lugares diferentes dos erros dos *boosters* — mas isso só funcionou porque os pesos foram otimizados (não uniformes), permitindo ao modelo fraco contribuir sem dominar.
6. **Melhorias na validação cruzada nem sempre se traduzem no teste real**: a variante multi-*seed* dos pesos otimizados melhorou o RMSPE de CV (0,2199) mas piorou no Kaggle (0,2398), evidenciando *overfitting* ao conjunto *out-of-fold* quando o espaço de otimização (14 modelos) é grande demais em relação aos dados disponíveis.
7. **O número ótimo de *trials* do Optuna depende do tamanho do conjunto de dados**: no Trabalho 1 (20.000 amostras), buscas mais longas pioraram a generalização; aqui (4.632 amostras), buscas mais longas melhoraram — não existe uma regra universal, e a validação empírica em cada problema é indispensável.

## Referências

- [1] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.
- [2] Ke, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems (NeurIPS)*, v. 30, 2017.
- [3] Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*, p. 785–794, 2016.
- [4] Prokhorenkova, L. et al. CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems (NeurIPS)*, v. 31, 2018.
- [5] Akiba, T. et al. Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD*, p. 2623–2631, 2019.
- [6] Friedman, J. H. Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, v. 29, n. 5, p. 1189–1232, 2001.
- [7] Breiman, L. Random Forests. *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.
- [8] Micci-Barreca, D. A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems. *ACM SIGKDD Explorations Newsletter*, v. 3, n. 1, p. 27–32, 2001.
- [9] Hastie, T.; Tibshirani, R.; Friedman, J. *The Elements of Statistical Learning*. 2. ed. New York: Springer, 2009.
- [10] McKinney, W. Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, p. 56–61, 2010.
- [11] Virtanen, P. et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, v. 17, p. 261–272, 2020.